

DP Mastery

8 patterns. ~50 problems. Pattern-first FAANG-prep cheatsheet.

[@rathoreadiitya](#) - Instagram

v1.0 · Updated 2026-05-22

Why this sheet exists

DP is the topic that breaks the most candidates. Not because it's hard - because they treat it as “dozens of tricks” instead of 7 recurring shapes. The bar at FAANG is: recognise the shape, write the recurrence, code it. Five minutes of pattern matching beats five hours of “maybe I'll memoise harder.”

This sheet collapses DP into 8 patterns: 1D linear, Knapsack family, LIS, LCS / Edit Distance, Grid, Interval, Subsequence / partition / state-machine, and Bitmask DP. Each pattern has a mental model, code walkthrough, traps, warm-up, and 3-4 LC problems in the exact order to solve them. Own these 8 and you cover the vast majority of DP problems asked at FAANG.

How to use: Crash course → 5 min. Decision tree → internalise it. Patterns 1-8 in order - each has warm-up, walkthrough(s), traps, and curated practice. Cross-pattern traps → read once before any interview. Revision drill → 20 minutes the night before.

Inside this sheet

1. Crash course (5 min)
2. Pattern decision tree (problem → pattern in 10 seconds)
3. Pattern 1 - 1D linear DP
4. Pattern 2 - Knapsack family (0/1 + unbounded + subset sum)
5. Pattern 3 - LIS
6. Pattern 4 - LCS / Edit Distance (2D string DP)
7. Pattern 5 - Grid DP
8. Pattern 6 - Interval DP
9. Pattern 7 - Subsequence / partition / state-machine
10. Pattern 8 - Bitmask DP (subsets-as-state, $N \leq \sim 20$)
11. Cross-pattern traps (read once before any interview)
12. Pre-interview revision drill (20 min)
13. 7-day study plan

Crash course (5 min)

Every DP problem reduces to four questions, in this exact order:

14. STATE: What does $dp[i]$ (or $dp[i][j]$) mean? Write it in one sentence. If you can't, you don't have a DP yet.
15. TRANSITION: How is $dp[i]$ built from earlier states? Write the recurrence.

16. BASE: What are $dp[0]$ / $dp[1]$ / $dp[0][j]$ / $dp[i][0]$? Hand-verify on the smallest case.
17. ORDER: In what order do I fill the table so every read sees a written cell? Iterate accordingly.
- If all four answers are clean, the code is mechanical. If any one is fuzzy, you'll burn 40 minutes debugging.

Big insight you keep missing: a DP problem is NOT a different algorithm per question. It's the SAME four-step skeleton over and over, with different state shapes. Once you can name the shape (1D / 0/1 knapsack / LCS / grid / interval / state-machine / bitmask), the code writes itself.

Pattern decision tree (problem → pattern in 10 seconds)

What the problem says	Pattern
"Climbing stairs" / "decode ways" / "rob houses" / "max non-adjacent"	1 - 1D linear DP
"Pick subset with sum / weight / capacity" / "partition equal"	2 - 0/1 knapsack / subset-sum
"Coin change" / "unlimited use" / "num ways to make X"	2 - Unbounded knapsack
"Longest increasing" / "number of LIS" / "Russian Doll Envelopes"	3 - LIS
"Longest common subsequence" / "edit distance" / "distinct subsequences"	4 - LCS / Edit Distance
"Min / max path sum in grid" / "unique paths" / "maximal square"	5 - Grid DP
"Burst balloons" / "merge stones" / "min cost to cut" / "palindrome cuts"	6 - Interval DP
"Word break" / "regex match" / "stock buy/sell with state"	7 - State machine / partition
"Visit all N nodes" / "assign tasks" / " $N \leq 16$ + subset state"	8 - Bitmask DP

The 8 patterns

Pattern 1 — 1D linear DP - "decision per index" (Fib family)

One-liner: State = $dp[i]$ depends on a SHORT trailing window ($dp[i-1]$, $dp[i-2]$, maybe $dp[i-3]$). Build left-to-right; rolling variables collapse memory to $O(1)$.

Mental model

Mental model: Walk the index once. At each i , the answer for $[0..i]$ is a function of the answer at $i-1$ and

i-2 (rarely further). Write the recurrence on paper before touching code - every 1D-DP bug comes from a wrong recurrence.

State design: $dp[i]$ = “the answer ENDING AT (or USING) index i ”. Final answer is usually $dp[n-1]$, or sometimes $\max(dp)$ / $\sum(dp)$.

Memory collapse: If $dp[i]$ only reads $dp[i-1]$ and $dp[i-2]$, you don't need the array - just two rolling variables ($prev$, $prev2$). Saves $O(N)$ space and impresses interviewers.

Invariant: Once $dp[i]$ is set, it's the optimal value for the subproblem ending at i . No revisits.

Spot signals

- “Climbing stairs” / “ways to decode” / “ways to tile” - counting versions of Fibonacci.
- “House robber” / “max non-adjacent sum” - optimisation versions.
- “Min / max cost ending at index i ” - maintenance problem.
- Anything with a “pick or skip” / “step 1 or step 2” binary choice per position.

Walkthrough - LC 198 House Robber ($O(1)$ space)

At each house, choose: rob it ($prev2 + nums[i]$) or skip ($prev$). Two rolling variables, one pass.

Python

```
def rob(nums):
    prev2, prev1 = 0, 0
    for n in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + n)
    return prev1
```

Walkthrough - LC 213 House Robber II (circular)

Houses form a circle, so the first and last can't both be robbed. Run House Robber TWICE: on $nums[:-1]$ and $nums[1:]$; take the max.

Python

```
def rob_circle(nums):
    if len(nums) == 1: return nums[0]
    def lin(arr):
        p2, p1 = 0, 0
        for n in arr:
            p2, p1 = p1, max(p1, p2 + n)
        return p1
    return max(lin(nums[:-1]), lin(nums[1:]))
```

Common traps

- Off-by-one at the BASE case. Always hand-check $dp[0]$ and $dp[1]$ before running. The single most common 1D-DP bug.
- Allocating $dp[]$ of size n instead of $n+1$ when the recurrence reads $dp[i-2]$ from $i = 0$. Either pad with sentinels or guard with $\max(0, i-2)$.
- Forgetting that $dp[i]$ in House Robber is the BEST UP TO i , not the value FROM robbing i . Two different semantics; pick one and stick to it.
- Trying to memory-collapse before the recurrence is correct. First make it work with $dp[]$, then collapse - not the other way round.

Warm-up first — build the mechanics

Solve this cold before LC 198. LC 70 is pure Fibonacci with a different name - locks the “write recurrence + iterate + return $dp[n]$ ” rhythm before any decision logic enters.

#	Problem	Difficulty	Link
70	Climbing Stairs - Fibonacci in disguise; the simplest possible 1D DP	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	198	House Robber - the canonical “pick or skip”; foundation for the whole family	Medium	LeetCode
2	213	House Robber II - circular variant; run-twice trick	Medium	LeetCode
3	746	Min Cost Climbing Stairs - same skeleton, minimising instead of counting	Easy	LeetCode
4	91	Decode Ways - 1D DP with two-character lookahead AND zero edge cases; the hardest “easy” DP	Medium	LeetCode

Pattern 2 — Knapsack family - 0/1, unbounded, subset sum

One-liner: Given items with weights / values and a capacity, decide which to pick.
 $dp[i][w]$ = best value using first i items with capacity w . Almost every “sum equals X / minimum coins” problem is a knapsack remix.

Mental model

0/1 knapsack: Each item used AT MOST ONCE. Recurrence: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i]] + val[i])$. Iterate items outer, capacity inner; in the 1D-collapsed form, iterate capacity BACKWARDS to prevent reusing the same item.

Unbounded knapsack: Each item used UNLIMITED times. Same recurrence but in the 1D form, iterate capacity FORWARDS - so $dp[w - coin]$ sees the UPDATED dp (allowing reuse).

Subset sum / partition: Special case of 0/1 with all weights = values. $dp[w] = \text{True}$ iff some subset sums to w . Coin change is unbounded knapsack with min count / num ways.

Invariant: After processing item i , $dp[w]$ = optimal value achievable with exactly the first i items and capacity $\leq w$.

Spot signals

- “Pick a subset with sum / value / weight constraint”
- “Coin change” / “min coins to make X” / “number of ways to make X”
- “Partition equal subset sum” / “target sum with + / -”
- “Can I exactly fill capacity” / “max value within budget”

Walkthrough - LC 416 Partition Equal Subset Sum (1D 0/1)

Total sum S - if odd, impossible. Otherwise, can we make $S/2$? Subset-sum DP. Iterate capacity BACKWARDS to avoid double-counting an item.

Python

```
def canPartition(nums):
    s = sum(nums)
    if s % 2: return False
    target = s // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for n in nums:
        for w in range(target, n - 1, -1):           # backwards = each item
            once                                     once
            dp[w] = dp[w] or dp[w - n]
    return dp[target]
```

Walkthrough - LC 322 Coin Change (unbounded)

Min coins to make amount. Forward sweep on capacity = each coin can be reused.

Python

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for c in coins:
        for w in range(c, amount + 1):           # FORWARDS = reuse allowed
            if dp[w - c] + 1 < dp[w]:
                dp[w] = dp[w - c] + 1
    return dp[amount] if dp[amount] != float('inf') else -1
```

Common traps

- Iterating capacity FORWARDS in 0/1 knapsack. Silently turns it into unbounded - same item gets used many times.
- Iterating capacity BACKWARDS in unbounded. Silently turns it into 0/1 - you miss reuse and get wrong min/count.
- Forgetting $dp[0]$ = base case (True for subset-sum, 0 for coin-min, 1 for count-ways). The whole recurrence cascades off this.
- Coin Change (min count) confused with Coin Change II (number of ways). Two different problems; the OUTER loop matters - coins outer = ways (each combination counted once); coins inner = permutations.
- Off-by-one on the inner-loop bounds: `for w in range(c, amount+1)` (forward) vs `for w in range(target, n-1, -1)` (backward).

Warm-up first — build the mechanics

Solve this cold before LC 416. LC 1049 is the partition-into-two-near-equal-halves version stripped to its core: reduce a pile of stones by smashing pairs, minimise the leftover. Same subset-sum-on-half(sum) DP, simpler controls.

#	Problem	Difficulty	Link
1049	Last Stone Weight II - partition into two halves with min difference; the cleanest subset-sum warm-up	Medium	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	322	Coin Change - the canonical unbounded knapsack (min count); bar-raiser foundation	Medium	LeetCode
2	416	Partition Equal Subset Sum - 0/1 subset-sum; mandatory FAANG problem	Medium	LeetCode

Step	#	Problem	Difficulty	Link
3	494	Target Sum - + / - assignment, reduces to subset-sum-count	Medium	LeetCode
4	518	Coin Change II - number of WAYS (combinations, not min); coins outer = combinations	Medium	LeetCode
5	279	Perfect Squares - unbounded coin-change on perfect-square coins; classic 1D unbounded knapsack	Medium	LeetCode

Pattern 3 — LIS - Longest Increasing Subsequence (and friends)

One-liner: Find the longest strictly-increasing subsequence. $O(N^2)$ DP is the floor; $O(N \log N)$ patience sort with binary search is the speed.

Mental model

$O(N^2)$ DP: $dp[i]$ = LIS ending exactly at index i . For each i , scan $j < i$; if $nums[j] < nums[i]$, $dp[i] = \max(dp[i], dp[j] + 1)$. Final answer = $\max(dp)$. The intuition: every LIS ending at i extends some shorter LIS ending earlier.

$O(N \log N)$ patience: Maintain `tails` = the smallest possible tail value for an LIS of each length. For each new num, binary-search for the first tail $\geq$ num and overwrite. Length of tails at end = LIS length. `tails` is NOT itself a valid LIS - it just tracks lengths.

Invariant (patience): $tails[k]$ is always the smallest tail of ANY increasing subsequence of length $k+1$ seen so far. Smaller tails are easier to extend.

LIS $\rightarrow$ LDS $\rightarrow$ LCS bridge: LIS on negated array = longest decreasing. $LIS(nums) = LCS(nums, sorted_unique(nums))$ - useful when proving / generalising.

Spot signals

- “Longest increasing / decreasing subsequence” verbatim.
- “Russian Doll Envelopes” / 2D LIS after a sort trick.
- “Minimum number of patches / arrows to make non-decreasing” - LIS-style greedy.
- “Number of LIS” / “Longest bitonic” - LIS variants requiring two passes.

Walkthrough - LC 300 LIS, $O(N \log N)$

Patience-sort: for each num, replace the first tail $\geq$ num with num. tails grows only when num is bigger than ALL tails.

Python

```
from bisect import bisect_left

def lengthOfLIS(nums):
    tails = []
    for n in nums:
        i = bisect_left(tails, n)
        if i == len(tails): tails.append(n)
        else: tails[i] = n
    return len(tails)
```

Common traps

- Using bisect_RIGHT instead of bisect_LEFT for strictly increasing. Right would allow duplicates. Use right for “non-decreasing” / “strictly INCR” rule swap.
- Treating `tails` as a real LIS. It isn't - only its LENGTH is correct. If you need to reconstruct the LIS, store parent pointers with the $O(N^2)$ DP version.
- Forgetting the equals-case for “non-decreasing” vs “strictly increasing”. Read the problem twice and choose bisect_left vs bisect_right accordingly.
- Russian Doll Envelopes: sorting by width ASC and height ASC is WRONG - same-width envelopes would chain. Sort width ASC, height DESC so same-width pairs can't both go into LIS on heights.

LIS has no Easy LC warm-up. Do the $O(N^2)$ DP cold first (loop within loop), then memorise the 6-line $O(N \log N)$ patience version above. Once you can derive both, every LIS variant collapses to “what's the comparator?”

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	300	Longest Increasing Subsequence - the canonical; $O(N \log N)$ is the interview-grade answer	Medium	LeetCode
2	354	Russian Doll Envelopes - sort + LIS on second dimension; classic 2D trick	Hard	LeetCode
3	673	Number of LIS - LIS with counts; two parallel DP arrays	Medium	LeetCode
4	368	Largest Divisible Subset - sort then LIS on divisibility; reconstruct via parent pointers	Medium	LeetCode

Pattern 4 — LCS / Edit Distance - 2D string DP

One-liner: Two strings, one 2D table. $dp[i][j]$ = answer for $s1[:i]$ vs $s2[:j]$. Transition compares $s1[i-1]$ and $s2[j-1]$; if match, extend; otherwise take the best of the three neighbours.

Mental model

Mental model: Build a $(m+1) \times (n+1)$ table with $dp[0][*]$ and $dp[*][0]$ as base cases. Fill row-by-row. The recurrence is one of two patterns: “match $\rightarrow dp[i-1][j-1] + 1$; mismatch $\rightarrow$ best of $dp[i-1][j]$, $dp[i][j-1]$, $(dp[i-1][j-1])$ ” depending on the problem.

Memory collapse: Only two rows are ever active. Replace $dp[m+1][n+1]$ with $prev[n+1] + curr[n+1]$. Saves $O(MN)$ space.

Edit distance vs LCS: LCS = max chars in common. Edit distance = min ops to transform. LCS uses max + match-bonus; Edit distance uses min + mismatch-cost. Same table shape, dual recurrence.

Invariant: $dp[i][j]$ is the optimal answer for the first i chars of $s1$ and first j chars of $s2$ - nothing about characters beyond.

Spot signals

- Two strings, one question - “longest common”, “min operations”, “distinct subsequences”, “interleaving”.
- “Minimum edit distance” / “one edit away” - literal Levenshtein.
- “Delete operation for two strings” - reduces to LCS.
- “Is $s3$ an interleaving of $s1$ and $s2$ ” - boolean 2D DP.

Walkthrough - LC 1143 LCS

Match $\rightarrow dp[i][j] = dp[i-1][j-1] + 1$. Mismatch $\rightarrow$ max of $dp[i-1][j]$ and $dp[i][j-1]$.

Python

```
def longestCommonSubsequence(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

Walkthrough - LC 72 Edit Distance

3-way min: insert ($dp[i][j-1]+1$), delete ($dp[i-1][j]+1$), replace ($dp[i-1][j-1]+1$). Match: just inherit $dp[i-1][j-1]$.

Python

```
def minDistance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]
```

Common traps

- Confusing index i in dp with index i in the string - dp uses 1-based positions ($s1[i-1]$). Always $s1[i-1] / s2[j-1]$, never $s1[i] / s2[j]$.
- Forgetting the base row / column. $dp[0][j] = j$ for edit distance (cost to insert j chars), $dp[0][j] = 0$ for LCS. They are NOT the same.
- Edit distance min over only 2 of the 3 neighbours. All three operations must be considered.
- Memory-collapsing too early. Get the $(m+1) \times (n+1)$ version right first, then swap to two rows.
- Distinct Subsequences (LC 115): the recurrence has a $+$ (not $\max$) in the matching branch and inherits from $dp[i-1][j]$ regardless. Different shape - don't paste LCS code.

Warm-up first — build the mechanics

Solve this cold before LC 72. LC 583 is edit distance with ONLY delete allowed - reduces to $m + n - 2 * \text{LCS}$. Locks the 2D-table-fill rhythm in one of its simplest forms.

#	Problem	Difficulty	Link
583	Delete Operation for Two Strings - LCS-based; the simplest two-string DP	Medium	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	1143	Longest Common Subsequence - the canonical 2D-string DP; mandatory FAANG	Medium	LeetCode
2	72	Edit Distance - textbook Levenshtein; bar-raiser at Google	Medium	LeetCode
3	115	Distinct Subsequences - count, not max; recurrence shape differs from LCS - re-derive it	Hard	LeetCode
4	97	Interleaving String - boolean 2D DP; alternative $O(M+N)$ memo on string slices works too	Medium	LeetCode
5	44	Wildcard Matching - 2D boolean DP with '?' and '*'; cleaner cousin of LC 10 regex	Hard	LeetCode
6	718	Maximum Length of Repeated Subarray - LCS for CONTIGUOUS match (so reset on mismatch); subarray-LCS hybrid	Medium	LeetCode
7	1092	Shortest Common Supersequence - LCS to find common backbone, then merge the rest; output-reconstruction LCS	Hard	LeetCode

Pattern 5 — Grid DP - paths, sums, decisions on a 2D matrix

One-liner: Walk a grid in (usually) one direction. $dp[i][j]$ depends on $dp[i-1][j]$ and / or $dp[i][j-1]$. Boundary row / column are the base cases.

Mental model

Mental model: Decide the move set (usually right + down). $dp[i][j]$ is the optimal value to REACH (i, j) from start, OR the value GOING FROM (i, j) to end - pick one and stick to it. Iterate in topological order of the moves: row-by-row, left-to-right.

Memory collapse: Most grid DPs only read the previous row - collapse to a 1D array. For LC 64 Min Path Sum, you can even update in place if input is mutable.

Obstacles: Set $dp[i][j] = 0$ (count) or infinity (min) when $grid[i][j]$ is blocked. Update neighbours accordingly.

Invariant: $dp[i][j]$ is the optimal answer for the sub-rectangle ending at (i, j) under the chosen direction.

Spot signals

- “Unique paths” / “number of ways to reach (m, n) moving right / down”

- “Min / max path sum in a grid”
- “Triangle / dungeon game” - grid DP with custom move set or backward fill.
- “Maximal square / rectangle” - grid DP where dp encodes shape size.

Walkthrough - LC 64 Min Path Sum (in-place)

$dp[i][j] = grid[i][j] + \min(\text{top}, \text{left})$. First row / column degenerate to prefix sums.

Python

```
def minPathSum(grid):
    R, C = len(grid), len(grid[0])
    for i in range(R):
        for j in range(C):
            if i == 0 and j == 0: continue
            top = grid[i-1][j] if i else float('inf')
            left = grid[i][j-1] if j else float('inf')
            grid[i][j] += min(top, left)
    return grid[R-1][C-1]
```

Common traps

- Iterating in the wrong order. Right / down moves need top-to-bottom, left-to-right. Inverted order reads UNFILLED cells.
- Forgetting to seed the first row / column. $dp[0][0]$ is the move-set's anchor, and $dp[0][j]$ / $dp[i][0]$ are degenerate.
- Maximal Square (LC 221): the recurrence is $dp[i][j] = 1 + \min(\text{top}, \text{left}, \text{top-left})$ AT a '1' cell - the THREE neighbours are required, not two.
- Dungeon Game (LC 174): fill from the END backwards, not from the start. The constraint is on the minimum HP needed entering each cell, not accumulated cost.
- Mutating input grid when the problem says don't. Use a separate dp array - the in-place trick is a convenience, not always permitted.

Warm-up first — build the mechanics

Solve this cold before LC 64. LC 62 has zero values, no min / max - just count paths. Locks the grid-walk + boundary-base-case rhythm before any cost decisions enter.

#	Problem	Difficulty	Link
62	Unique Paths - pure count; the simplest grid DP	Medium	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	64	Minimum Path Sum - the canonical optimisation grid DP; mandatory FAANG	Medium	LeetCode
2	63	Unique Paths II - LC 62 with obstacles; tests the $dp[i][j] = 0$ on blocked-cell discipline	Medium	LeetCode
3	221	Maximal Square - dp encodes side length; the THREE-neighbour recurrence	Medium	LeetCode
4	120	Triangle - grid DP on a triangle; collapse to 1D bottom-up for $O(N)$ space	Medium	LeetCode
5	174	Dungeon Game - fill the grid in REVERSE from bottom-right; classic min-health-to-survive DP	Hard	LeetCode

Pattern 6 — Interval DP - “best split of a range”

One-liner: $dp[i][j]$ = optimal answer over the subarray $[i, j]$. Transition tries EVERY split point k in between. $O(N^3)$ is the default cost.

Mental model

Mental model: When a problem says “merge / burst / break / parenthesise this range optimally,” you need three loops: outer = interval LENGTH (smallest first!), middle = left endpoint i , inner = split point k between i and $j = i + \text{length} - 1$. Smaller intervals MUST be solved before larger ones.

Recurrence shape: $dp[i][j]$ = best over k in $[i, j-1]$ of $f(dp[i][k], dp[k+1][j], \text{merge_cost}(i, k, j))$.

Why length-first: $dp[i][j]$ reads $dp[i][k]$ and $dp[k+1][j]$ where both $k+1 \leq j$ and $i \leq k$ - so both sub-intervals are SHORTER. If you iterate i / j in any other order, you'll read unfilled cells.

Burst-balloons twist: Reverse the framing - $dp[i][j]$ = max coins ASSUMING balloons $i..j$ are burst LAST. Then k is the LAST balloon burst inside (i, j) , and its neighbours are $\text{coins}[i-1]$ and $\text{coins}[j+1]$ (the boundary, not interior).

Spot signals

- “Burst balloons” / “remove boxes” - reverse-thinking variant.
- “Matrix chain multiplication” / “minimum cost to merge stones / files”
- “Palindrome partitioning II” / “minimum cuts” (boundary-style interval DP).
- “Optimal BST cost” / “strange printer” - range-split DP.

Walkthrough - LC 312 Burst Balloons (reverse interval DP)

Pad the array with 1s on both sides. $dp[i][j]$ = max coins from bursting all balloons STRICTLY inside (i, j) . For each split k , $dp[i][j] = \max(dp[i][j], dp[i][k] + \text{nums}[i] * \text{nums}[k] * \text{nums}[j] + dp[k][j])$.

Python

```
def maxCoins(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):                # interval size on padded
    array                                     array
        for i in range(0, n - length):
            j = i + length
            for k in range(i + 1, j):          # k = last balloon burst in
    (i,j)                                     (i,j)
                cand = dp[i][k] + nums[i]*nums[k]*nums[j] + dp[k][j]
                if cand > dp[i][j]: dp[i][j] = cand
    return dp[0][n-1]
```

Common traps

- Iterating i and j directly instead of by INTERVAL LENGTH. Reads unfilled subintervals - silently wrong.
- Burst Balloons: the natural “WHICH balloon to burst FIRST” leaves nondeterministic neighbours. Reverse it to “LAST” - then the neighbours are fixed at the boundary.
- Off-by-one on split-point k . The valid range is k in $[i, j-1]$ (or $[i+1, j-1]$ for boundary-style); always sanity-check on a length-2 case.
- Greedy split on “always pick the smallest first” - wrong for non-additive merge costs (LC 1000 Stones).
- $O(N^3)$ memory pressure: $dp[N][N]$ integers - if N is big, you're in trouble. Most interval-DP problems cap N at a few hundred for that reason.

Interval DP has no truly Easy LC warm-up. Solve LC 5 (Longest Palindromic Substring) DP version first - it's interval DP with a boolean table (“is $s[i..j]$ a palindrome?”), the cleanest entry point to the length-first iteration order.

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	5	Longest Palindromic Substring - boolean interval DP; the easiest interval DP	Medium	LeetCode

Step	#	Problem	Difficulty	Link
2	1547	Minimum Cost to Cut a Stick - textbook interval DP; great length-first warmup	Hard	LeetCode
3	312	Burst Balloons - the famous reverse-interval-DP problem; bar-raiser at Google	Hard	LeetCode
4	132	Palindrome Partitioning II - min cuts; mixed boolean + linear DP	Hard	LeetCode
5	516	Longest Palindromic Subsequence - interval DP on s vs reversed(s); the cleanest non-substring variant	Medium	LeetCode
6	96	Unique Binary Search Trees - Catalan via interval DP; sum over root choices for each interval length	Medium	LeetCode
7	887	Super Egg Drop - K eggs, N floors min trials; classic interval DP, then optimise via inverse formulation	Hard	LeetCode

Pattern 7 — DP on subsequences / partition / state-machine

One-liner: When the choice at each index is multi-way OR the state needs extra dimensions (cash, holding stock, last action), use a state-machine DP. $dp[i][state]$ for each step.

Mental model

State-machine framing: List the distinct STATES (holding / not-holding / cooldown / etc). For each (i, state), write the transitions: which (i+1, state') can we move to and at what gain? Then $dp[i][state]$ = best value ending at index i in that state.

Word-break style: $dp[i]$ = True iff $s[:i]$ can be segmented. For each i, scan $j < i$: $dp[i] = \text{any}(dp[j] \text{ and } s[j:i] \text{ in dictionary})$. $O(N^2)$ with the dictionary turned into a set.

Subsequence-count style: $dp[i]$ = number of ways to do X using exactly i items. Watch the duplicate-handling: the outer loop choice (items vs targets) determines whether you count permutations vs combinations.

Invariant: $dp[i][state]$ is the optimal answer for the prefix ending at i AND being in the named state - independent of choices made beyond i.

Spot signals

- “Best time to buy and sell stock” with k transactions / cooldown / fee - the iconic state-machine family.
- “Word break” / “concatenated words” / “split into dictionary words”

- “Paint house” / “min cost where adjacent decisions interact”
- “Match regex / wildcard” - boolean state-machine 2D DP.
- “Count unique characters / palindromic subsequences” - partition-by-character DP.

Walkthrough - LC 309 Best Time to Buy/Sell with Cooldown (state machine)

3 states per day: holding, sold (must cooldown tomorrow), rested. Transitions: rest $\rightarrow$ rest or buy; hold $\rightarrow$ hold or sell; sold $\rightarrow$ rest only.

Python

```
def maxProfit(prices):
    hold, sold, rest = float('-inf'), 0, 0
    for p in prices:
        prev_sold = sold
        sold = hold + p                # sell today
        hold = max(hold, rest - p)    # keep or buy today
        rest = max(rest, prev_sold)   # rest, or yesterday's sold
    return max(sold, rest)
```

Walkthrough - LC 139 Word Break (subsequence DP)

$dp[i]$ = True iff $s[:i]$ segments into dictionary words. For each i , scan $j < i$.

Python

```
def wordBreak(s, wordDict):
    words = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

Common traps

- Stock state-machine: forgetting to capture `prev_sold` BEFORE updating `sold`. Cooldown breaks otherwise.
- Word Break: building the inner string slice $s[j:i]$ inside the loop is $O(N)$ per slice - if N is big, use a Trie or memoise to keep total under $O(N^2)$.

- Coin Change vs Coin Change II: outer-loop choice (coins vs amounts) is the WHOLE difference. Coins outer = combinations; amounts outer = permutations.
- Regex / Wildcard DP (LC 10 / 44): the recurrence has 4 cases (match-char, '.', '?', '*') - derive them on paper before coding.
- Conflating “subsequence” (non-contiguous) and “subarray / substring” (contiguous). Different DP shapes. Read the problem TWICE.

Warm-up first — build the mechanics

Solve this cold before LC 309. LC 121 is the original single-transaction stock problem - just track min-so-far. Locks the single-pass profit-tracking rhythm before adding states for cooldown.

#	Problem	Difficulty	Link
121	Best Time to Buy and Sell Stock - single transaction; the simplest stock DP	Easy	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	139	Word Break - the canonical subsequence DP; mandatory FAANG	Medium	LeetCode
2	309	Best Time to Buy/Sell with Cooldown - 3-state state machine; iconic stock variant	Medium	LeetCode
3	188	Best Time to Buy/Sell IV - k transactions; full 2D state machine	Hard	LeetCode
4	10	Regular Expression Matching - boolean 2D DP with 4-case transition; classic Google interview	Hard	LeetCode
5	122	Best Time to Buy/Sell II - unlimited transactions; greedy-equivalent state machine warm-up	Medium	LeetCode
6	123	Best Time to Buy/Sell III - at most 2 transactions; 4-state DP (buy1/sell1/buy2/sell2)	Hard	LeetCode
7	714	Best Time to Buy/Sell with Fee - unlimited with per-sale cost; 2-state hold/cash DP	Medium	LeetCode
8	337	House Robber III - tree DP returning (rob, skip) per node; rob/skip state-machine on a binary tree	Medium	LeetCode

Pattern 8 — Bitmask DP (subsets-as-state when N is tiny)

One-liner: When $N \leq \sim 20$ and the state is “which subset of items did I use,” encode the subset as an N-bit integer. $dp[mask]$ = best answer for the items represented by mask.

Mental model

When to reach for it: $N \leq \sim 16$ (sometimes 20) AND the natural state is “which items are done.” 2^N states fit in memory only because N is small. If $N > \sim 22$, bitmask DP is too expensive - look for a different pattern.

State: $dp[mask]$ (1D) or $dp[mask][i]$ (2D, when current position also matters). $mask$ = bitmask over N items where bit $k = 1$ means item k is used / visited / placed.

Transition: From $dp[mask]$, try every item k NOT in $mask$. New $mask = mask | (1 \ll k)$. Update $dp[new_mask]$ using whatever cost function the problem defines.

Submask enumeration trick: Iterate every submask of $mask$: $sub = mask$; while $sub > 0$: ...; $sub = (sub - 1) \& mask$. Touches each submask exactly once - total work across all masks is $O(3^N)$, not $O(4^N)$.

Bit tricks: $mask \& (1 \ll k)$ checks if bit k is set. $bin(mask).count('1')$ counts set bits. $(mask \& -mask)$ isolates the lowest set bit. $\sim mask \& full$ enumerates the COMPLEMENT subset.

Spot signals

- “ $N \leq 12 / 15 / 16 / 20$ ” - the constraint itself screams bitmask.
- “Visit ALL nodes / cities” with small N (TSP family).
- “Partition the array into K equal-sum groups” with small N .
- “Assign tasks to people” / “sessions of fixed duration” with small task count.
- “Form a valid arrangement / permutation” - $dp[mask]$ = ways to fill the first $popcount(mask)$ positions.

Walkthrough - LC 526 Beautiful Arrangement (bitmask over chosen numbers)

$dp[mask]$ = number of valid arrangements using exactly the numbers indicated by $mask$. The next position to fill = $popcount(mask)$. Number k (1-indexed) can sit at position p if $k \% p == 0$ OR $p \% k == 0$.

Python

```
def countArrangement(n):
    dp = [0] * (1 << n)
    dp[0] = 1
    for mask in range(1, 1 << n):
        pos = bin(mask).count('1')          # next position to fill (1-
indexed)
        for k in range(1, n + 1):          # try placing number k at
this position
            if mask & (1 << (k - 1)) and (k % pos == 0 or pos % k == 0):
                dp[mask] += dp[mask ^ (1 << (k - 1))]
    return dp[(1 << n) - 1]
```

Walkthrough - LC 847 Shortest Path Visiting All Nodes (TSP-flavored BFS)

State = (node, mask) where mask = nodes visited so far. BFS from every node as starting point; first time we pop a state with mask == $(1 \ll n) - 1$ the distance is optimal.

Python

```
from collections import deque

def shortestPathLength(graph):
    n = len(graph)
    full = (1 << n) - 1
    q = deque()
    seen = set()
    for i in range(n):
        q.append((i, 1 << i, 0))
        seen.add((i, 1 << i))
    while q:
        node, mask, d = q.popleft()
        if mask == full: return d
        for nei in graph[node]:
            nm = mask | (1 << nei)
            if (nei, nm) not in seen:
                seen.add((nei, nm))
                q.append((nei, nm, d + 1))
    return 0
```

Common traps

- Allocating 2^N when N is 25 - that is 32M states, blown memory. Bitmask DP is ONLY for $N \leq 20$. If N is bigger, the problem wants a different approach.
- Off-by-one between “k-th item” and “k-th bit”. Items are typically $1..N$ but bits are $0..N-1$. Use $(k - 1)$ inside the shift.
- Iterating items linearly when the recurrence is over SUBSETS of mask. The submask trick $(sub - 1) \& mask$ is the right loop, not a for-k loop.
- Wrong iteration order: $dp[mask]$ depends on $dp[smaller\ mask]$, so iterate $mask = 0..2^N - 1$ ascending. Descending or random order silently corrupts answers.
- Memoising with `@lru_cache(maxsize=None)` on a 5+ argument function - still correct but much slower than an explicit `dp[][]` table. For tight TLs, prefer the table.

Warm-up first — build the mechanics

Solve LC 526 cold before LC 847. It is the smallest possible bitmask DP - just permutation counting with a constraint. It locks the `dp[mask]` indexing and the popcount-as-position trick before adding graph edges on top in LC 847.

#	Problem	Difficulty	Link
526	Beautiful Arrangement - dp[mask] permutation count; the gentlest bitmask DP on LeetCode	Medium	LeetCode

Practice

Solve in this order — each problem unlocks the next.

Step	#	Problem	Difficulty	Link
1	698	Partition to K Equal Sum Subsets - dp[mask] feasibility; can mask be split into K equal-sum groups	Medium	LeetCode
2	847	Shortest Path Visiting All Nodes - TSP-flavored BFS over (node, mask); the iconic bitmask + graph problem	Hard	LeetCode
3	1986	Minimum Number of Work Sessions to Finish the Tasks - dp[mask] = min sessions to cover task subset	Medium	LeetCode
4	691	Stickers to Spell Word - dp[mask] over remaining target letters; BFS by sticker count; bar-raiser at Google	Hard	LeetCode

Cross-pattern traps (read once before any interview)

Trap	Pattern most at risk	How to dodge
Skipping the 4-step contract (state / transition / base / order)	all	Write all four on paper BEFORE coding. Even one fuzzy answer costs 40 minutes of debugging.
0/1 vs unbounded knapsack direction confusion	2	0/1 = capacity BACKWARDS (each item once). Unbounded = capacity FORWARDS (allows reuse).
bisect_left vs bisect_right in LIS	3	Strictly increasing → bisect_left. Non-decreasing → bisect_right.
Treating LCS like Edit Distance (or vice versa)	4	LCS = max with match-bonus. Edit distance = min with 3-way ops. Different recurrences.
Iterating i, j directly in interval DP instead of by length	6	Outer loop MUST be interval length, smallest first.
Burst Balloons “first” framing	6	Use “LAST balloon burst inside” framing. Boundary neighbours become deterministic.
State-machine: forgetting to capture prev before update	7	Stock with cooldown is the classic - prev_sold MUST be cached before sold is rewritten.
Coin Change I vs II: outer-loop ambiguity	2, 7	Coins outer = combinations (LC 518). Amounts outer = permutations. Match the question.

Pre-interview revision drill (20 minutes)

Run this the night before:

18. 30 seconds per pattern, name the state shape out loud (all 8).
19. 3 minutes per pattern, write the recurrence on paper (no IDE).
20. 5 minutes, walk through LC 198 (House Robber) - if you can explain why $\text{prev2} + n$ vs prev1 is the choice, your 1D-DP is solid.
21. 5 minutes, walk through LC 1143 (LCS) - if you can derive the recurrence from scratch on a 3x3 table, your 2D-string DP is solid.
22. 2 minutes, name the #1 trap in each pattern (the cross-pattern table above).

If you fumble any of these → don't open more new problems, go re-do the warm-up + canonical problem for that pattern.

7-day study plan

Day	Focus	Problems	Notes
1	Pattern 1 (1D linear)	LC 70, 198, 213, 746, 91	Foundation day. By end of day, the rolling-variable rhythm should flow without thinking.
2	Pattern 2 (Knapsack)	LC 1049, 322, 416, 494, 518, 279	0/1 vs unbounded direction matters — derive both, code both. Coin Change II is the partner of LC 322; Perfect Squares (279) is the unbounded warm-up.
3	Pattern 3 (LIS) + Pattern 4 warm-up	LC 300, 354, 673, 583	Memorise the 6-line patience version. Russian Doll is the 2D twist.
4	Pattern 4 (LCS / Edit Distance)	LC 1143, 72, 115, 97	Two-row memory collapse after you nail the $(m+1) \times (n+1)$ version.
5	Pattern 5 (Grid DP)	LC 62, 63, 64, 221, 120	Maximal Square's THREE-neighbour recurrence is the trap to internalise.
6	Pattern 6 (Interval DP)	LC 5, 1547, 312, 132	Length-first iteration is the entire pattern. Burst Balloons is the boss.
7	Pattern 7 (state machine) + Pattern 8 (Bitmask DP) + revision	LC 121, 309, 139, 188, 526, 698, 847	Stock family + word break + bitmask. Bitmask = $N \leq 16$ + subset state. Combine + revision drill.

Total: ~35 anchor problems across 7 days (the table above is the spine). The full sheet has ~50 problems — once the day's anchors are solid, extend with the rest of each pattern's practice rows. If a problem takes > 30 minutes without progress, peek at the editorial, understand the recurrence, write the code from scratch the next day. NEVER copy-paste; the muscle is in the

typing.

The big idea

DP looks like 50 different problems. It's 8 patterns $\times$ 4-6 reps each. The students who clear FAANG don't memorise 50 solutions - they recognise which of 8 shapes the problem is in, write the 4-step contract, and code it.

Do the 50 must-do problems. Internalise the recurrences. Then every future DP problem is just a remix.

Pseudocode-first reading: every code block is shown in Python — read each as pseudocode and translate to your interview language (Java / C++ / TS / Go). Single-language keeps each sheet lean; the patterns travel.